

Problem 4, Cell 4: Sun's conjecture for all $k \leq 12$,
and a status report for $9 \leq k \leq 16$

Official statement. For integers a and $m \geq 1$ write $a \pmod{m} = \{a + mt : t \in \mathbb{Z}\}$. A finite family of classes $a_1 \pmod{m_1}, \dots, a_k \pmod{m_k}$ is pairwise disjoint if $a_i \pmod{m_i} \cap a_j \pmod{m_j} = \emptyset$ whenever $i < j$. Nothing is assumed about the moduli beyond $m_i \geq 1$: they may repeat, and the residues a_i may repeat as well. By the Chinese remainder theorem,

$$a_i \pmod{m_i} \cap a_j \pmod{m_j} \neq \emptyset \iff \gcd(m_i, m_j) \mid a_i - a_j. \quad (*)$$

Question. Let $k \geq 2$ and let $a_1 \pmod{m_1}, \dots, a_k \pmod{m_k}$ be pairwise disjoint. Must there be a pair $i < j$ with $\gcd(m_i, m_j) \geq k$?

Cell 4 (Every k up to 12, 5 points). A longer range, and a precise account of your method at the sizes just beyond it. Prove the statement for every $k \leq 12$. Then, for each k from 9 to 16 in turn, report exactly what your method leaves undecided at that k : if nothing, say so and prove it; if something, exhibit it in full. If that disagrees with any source you consulted, say which of the two is right and why. An answer that reports agreement with a source it did not test will be marked wrong.

Theorem 1. *Let $2 \leq k \leq 12$. If $a_1 \pmod{m_1}, \dots, a_k \pmod{m_k}$ are pairwise disjoint residue classes, then $\gcd(m_i, m_j) \geq k$ for some $i \neq j$.*

Structure of the proof.

- $k \leq 8$: by hand (Cell 3, `p4.c3.tex`, Theorem 1). It is not repeated here.
- $9 \leq k \leq 12$: by an exhaustive computation. Its certificate has five parts: the finite set and the proofs of the reductions (§1); the enumeration with every pruning rule proved (§2); the decision procedure (§3); node counts, wall clock and a rerun (§4); and the list of survivors, each decided (Appendix A and the attached files in `code/data/`).
- The code is `p4_dfs/sun_dfs.py`. It uses no SAT solver, and it was written independently of the SAT-based checker `p4_codex`.

Throughout, B denotes the gcd bound. For the conjecture at k , $B = k - 1$.

1 Reductions and the finite set

Lemma 2 (Criterion). *$a \pmod{m}$ and $b \pmod{n}$ are disjoint if and only if $\gcd(m, n) \nmid a - b$.*

Proof. If x is in both, then $a \equiv x \equiv b \pmod{g}$, where $g = \gcd(m, n)$. Conversely, if $a - b = gt$, take $um + vn = g$; then $x = a - umt = b + vnt$ is in both. $\square$

Lemma 3 (Normal form). *Let $a_i \pmod{m_i}$ ($i \in [k]$) be disjoint, $g_{ij} = \gcd(m_i, m_j)$, and $n_i = \text{lcm}_{j \neq i} g_{ij}$. Then:*

- $n_i \mid m_i$;
- $\gcd(n_i, n_j) = g_{ij}$ for all $i \neq j$;
- the classes $a_i \bmod n_i$ are pairwise disjoint;
- $n_i = \text{lcm}_{j \neq i} \gcd(n_i, n_j)$.

Proof. Each g_{ij} divides m_i , so $n_i \mid m_i$. Next, g_{ij} divides both n_i and n_j , so $g_{ij} \mid \gcd(n_i, n_j)$. Conversely, $\gcd(n_i, n_j) \mid \gcd(m_i, m_j) = g_{ij}$. Disjointness follows from Lemma 2, since the gcds are unchanged. The last identity is the definition of n_i , rewritten using $\gcd(n_i, n_j) = g_{ij}$. $\square$

We say that $(m_1, \dots, m_k)$ is in *normal form* if $m_i = \text{lcm}_{j \neq i} \gcd(m_i, m_j)$ for every i .

Lemma 4. *If (m_i) is in normal form and all $\gcd(m_i, m_j) \leq B$, then each m_i divides $L_B = \text{lcm}(1, \dots, B)$.*

Proof. Each m_i is the lcm of integers in $[1, B]$, and each of these divides L_B . $\square$

Lemma 5 (Density). *If $a_i \bmod m_i$ ($i \in [k]$) are disjoint, then $\sum_i 1/m_i \leq 1$.*

Proof. Let $M = \text{lcm}(m_1, \dots, m_k)$. Class i meets $[0, M)$ in exactly M/m_i integers, and these sets are disjoint. Hence $\sum_i M/m_i \leq M$. $\square$

Lemma 6 (Normal form, one prime at a time). *(m_i) is in normal form if and only if, for every prime p , the maximum of $v_p(m_1), \dots, v_p(m_k)$ is 0 or is attained at least twice.*

Proof. Write $e_i = v_p(m_i)$. Then $v_p(\text{lcm}_{j \neq i} \gcd(m_i, m_j)) = \max_{j \neq i} \min(e_i, e_j)$, and this equals e_i iff $e_i = 0$ or some $j \neq i$ has $e_j \geq e_i$. This holds for every i iff the maximum is 0 or attained at least twice. $\square$

Definition 7. $\mathcal{C}(k, B)$ is the set of multisets $\{m_1, \dots, m_k\}$ of divisors of L_B such that

- $2 \leq \gcd(m_i, m_j) \leq B$ for all $i \neq j$;
- the multiset is in normal form;
- $\sum_i 1/m_i \leq 1$.

Its elements are called *survivors*. A survivor is *SAT* if there are $a_1, \dots, a_k$ making the classes $a_i \bmod m_i$ pairwise disjoint, and *UNSAT* otherwise.

Proposition 8 (Finite reduction). *The conjecture holds at k if and only if every survivor in $\mathcal{C}(k, k-1)$ is UNSAT.*

Proof. A SAT survivor is a counterexample, by the first condition. Conversely, take a counterexample (a_i, m_i) with all $g_{ij} \leq k-1$ (and ≥ 2 , by Lemma 2). By Lemma 3 we get a disjoint family (a_i, n_i) in normal form with the same gcds. By Lemma 4, $n_i \mid L_{k-1}$, and by Lemma 5, $\sum 1/n_i \leq 1$. So $\{n_i\} \in \mathcal{C}(k, k-1)$, and it is SAT. $\square$

2 Enumeration of $\mathcal{C}(k, B)$

Encoding. Let $p_0 > p_1 > \dots > p_{r-1}$ be the primes $\leq B$ (largest first), and let $v_t = \lfloor \log_{p_t} B \rfloor = v_{p_t}(L_B)$. A k -tuple of divisors of L_B is a $k \times r$ matrix E with $0 \leq E_{it} \leq v_t$ and $m_i = \prod_t p_t^{E_{it}}$.

Lemma 9 (Canonical form). *Every multiset of k divisors of L_B is represented by exactly one matrix E whose rows are in lexicographically non-increasing order. Suppose the first t columns of E are fixed and their rows are lex non-increasing. Then rows with equal prefixes form contiguous blocks. The full matrix is lex non-increasing if and only if, recursively, column t is non-increasing within every block.*

Proof. Sorting the rows gives a representative. Two sorted matrices with the same multiset of rows are equal. The block statement is the definition of lexicographic order, applied one column at a time. $\square$

The enumerator fills E column by column and, within a column, row by row. When row i does not start a block, it only allows $E_{it} \leq E_{i-1,t}$. By Lemma 9, every multiset is produced exactly once. (Duplicates are also checked at run time.) It maintains the partial gcds $G_{ij} = \prod_{s \leq t} p_s^{\min(E_{is}, E_{js})}$ of the columns filled so far.

Filters during the search. None of these filters changes the output. Each can be switched off.

- (P1) *gcd bound* (`--no-gcd-prune`). G_{ij} divides the final $\gcd(m_i, m_j)$. So if $G_{ij} > B$, no completion is in $\mathcal{C}(k, B)$.
- (P2) *coprime pairs at the last prime* (`--no-coprime-prune`). Suppose that in the last column $E_{i,r-1} = 0$ and $G_{ij} = 1$ for some $j \neq i$. Then $\gcd(m_i, m_j) = 1$ in every completion. For $j < i$ this is immediate. For $j > i$, the missing factor is $p_{r-1}^{\min(0, E_{j,r-1})} = 1$.
- (P3) *forced-even rows* (`--no-forced-even-prune`). The primes are processed in decreasing order, so the last column is $p = 2$. Consider the moment when column $r - 2$ is complete. A row i with $G_{ij} = 1$ for some $j \neq i$ must have $E_{i,r-1} \geq 1$, since otherwise $\gcd(m_i, m_j) = 1$. Call such rows *forced*. If two forced rows $a \neq b$ have $2G_{ab} > B$, then $\gcd(m_a, m_b) \geq 2G_{ab} > B$ in every completion, so the subtree is empty. The test also runs inside column $r - 2$. Once rows $a, b \leq i$ have been set in that column, G_{ab} is final except for the factor from 2. So a partner $b \leq i$ with $G_{ab} = 1$ already makes a forced, and the test can be applied to the rows forced so far.
- (NF) *normal form per column* (`--late-nf` moves it to the leaves). By Lemma 6, normal form is a condition on each column separately. So it can be tested as soon as a column is complete.

At every leaf, all conditions defining $\mathcal{C}(k, B)$ are re-checked from the moduli:

- $2 \leq \gcd \leq B$ for every pair;
- normal form, recomputed from the definition;
- $\sum 1/m_i \leq 1$, tested exactly in integers as $\sum L_B/m_i \leq L_B$.

Toggle test. For $3 \leq k \leq 7$, all 2^4 combinations of the four toggles give identical survivor lists and an identical count at the “gcd in range and normal form” stage. The combinations with (P1) off are only run for $k \leq 6$, because without the gcd bound the tree grows too fast. For $8 \leq k \leq 12$, the results are also identical with (P3) on and off. (P3) shrinks the tree from 1 746 588 to 113 915 nodes at $k = 12$.

Oracle test. For $3 \leq k \leq 8$ ($B = k - 1$) and $3 \leq k \leq 6$ ($B = k$), the enumerator agrees with a naive enumeration of all multisets of divisors of L_B , filtered by the definition. The naive enumerator is a test oracle only.

3 Deciding each survivor

For a survivor $(m_1, \dots, m_k)$ we decide whether residues exist by backtracking.

- *Translation.* $a_i \mapsto a_i - c$ preserves disjointness, so the first class chosen is given $a = 0$.
- *Forward checking.* Each unplaced class i has a domain $D_i \subseteq \mathbb{Z}/m_i$. Placing $a_j = x$ deletes $\{y \in D_i : y \equiv x \pmod{g_{ij}}\}$. By Lemma 2, these are exactly the values that conflict with $a_j = x$. An empty domain means backtrack.
- *Interchangeability* (`--no-interchange`). Let $U_i = \text{lcm}\{g_{ij} : j \text{ unplaced}, j \neq i\}$, which divides m_i . If $x, x' \in D_i$ and $x \equiv x' \pmod{U_i}$, then x extends to a solution iff x' does. Both are compatible with the placed classes, since they lie in D_i . The constraint with an unplaced j depends only on the residue modulo g_{ij} , and $g_{ij} \mid U_i$. So one value per class of D_i modulo U_i suffices.
- *Order.* The next class is the one with the fewest candidates; ties go to the larger $\sum_j \log g_{ij}$. Correctness does not depend on the order.
- *Cross-checks.* `--plain-decide` is plain backtracking over all of $\mathbb{Z}/m_i$ with a static order, and with neither forward checking nor interchangeability.
 - All three deciders (default, `--no-interchange`, `--plain-decide`) agree on every survivor with $k \leq 10$.
 - The first two also agree on every survivor with $k = 11, 12, 13$.
- *SAT witnesses.* These are checked literally: for each pair, every element of one period of class i is tested for membership in class j , without using Lemma 2. With bound $B = k - 1$ no SAT survivor occurred.

Vacuity test. With bound $B = k$ ($k = 3, \dots, 6$), the same code finds SAT survivors, in particular $(k, \dots, k)$ with residues $0, 1, \dots, k - 1$. All SAT witnesses pass the literal check. So the pipeline is able to detect solutions.

4 Results

Machine: Apple Mac, 8 cores, CPython 3.12 + numpy, single-threaded. “SHA-256” is the hash of the canonical survivor list: a JSON list of ascending lists, sorted lexicographically, serialised compactly (`separators=(',', ' ', ':')`). The table shows the first 16 hex digits; the full values are in the JSON files.

k	L_{k-1}	enum. nodes	leaves	gcd+NF	survivors	UNSAT	dec. nodes	time (s)	SHA-256 (prefix)
3	2	3	1	1	0	0	0	0.00	4f53cda18c2baa0c
4	6	20	2	2	0	0	0	0.00	4f53cda18c2baa0c
5	12	40	6	6	0	0	0	0.00	4f53cda18c2baa0c
6	60	133	8	8	0	0	0	0.00	4f53cda18c2baa0c
7	60	341	47	47	2	2	13	0.00	01557c6ee0333625
8	420	1 263	71	71	7	7	42	0.01	1621baadfc46fc4b
9	840	2 557	329	329	6	6	53	0.03	febd71b7bbda2883
10	2520	6 093	570	570	3	3	25	0.07	b02fcc2277902756
11	2520	31 547	7 450	7 450	137	137	27 772	2.63	dee521791e4d6f13
12	27720	113 915	12 546	12 546	522	522	42 848	6.19	d7ab51816f731ab2
13	27720	288 390	55 827	55 827	4 395	4 395	3 550 936	283.45	2dbfd65f26961615
14	360360	3 217 925	139 820	139 820	19 880	19 880	6 272 555	1006.66	f2ec65da7dedb65c

(The SHA-256 of the empty list is `4f53cda1...`, which explains the repetition for $k \leq 6$.) The node and leaf counts are those of the default options. The last three count columns do not depend on the toggles.

Rerun. The table above comes from the original run (`p4_dfs/results/run_k03-13.log`, `run_k14.log`). A timed rerun on this laptop (26 September 2026, logs in `code/p4_crosscheck/timed_rerun/logs/`) repeated the full run, enumeration *and* decisions, for $k = 3, \dots, 13$ in 82.7s of total wall clock, and the enumeration alone for $k = 14$ (12.4s) and $k = 15$ (139.5s). Every count (nodes, leaves, stages, decision nodes), every survivor list and every SHA-256 was reproduced exactly. The `p4_dfs` *decisions* for $k = 14$ were not rerun; that row rests on the original run and on the independent `p4_codex` computation below. Earlier versions of the code, without (P3) and with rational density arithmetic, produced the same survivor lists and SHA-256 values for $k \leq 13$, with larger node counts.

Second implementation. The SAT-based checker `p4_codex` was written independently. It enumerates the same finite sets with its own code and decides each survivor with CaDiCaL 1.5.3 (via `python-sat 1.9.dev15`). The survivor lists of the two implementations were compared *elementwise* by `p4_crosscheck/compare_lists.py`, after canonical normalisation, for every $k = 3, \dots, 16$. The report is `compare_report.json`.

- The lists are identical for every $k \leq 16$: nothing is only in one list, and there are no duplicates.
- Their canonical SHA-256 values coincide. They are the values in the table above, and those in Section 5 for $k = 15, 16$.
- There is no decision disagreement on any survivor.

For $k \leq 12$ in particular, every survivor used in the proof below is decided UNSAT by both implementations.

Proof of Theorem 1. For $k \leq 8$: Cell 3. For $9 \leq k \leq 12$: by Proposition 8 it suffices that every survivor in $\mathcal{C}(k, k-1)$ is UNSAT. The enumeration of §2 produces all of $\mathcal{C}(k, k-1)$ (Lemma 9, filters (P1), (P2), (NF)). The decision of §3 returns UNSAT for each of the $6 + 3 + 137 + 522$ survivors. The survivors for $k = 9, 10$ are listed in Appendix A; those for $k = 11, 12$ are in `code/data/p4_dfs/k11_sun.json.gz` and `k12_sun.json.gz` (and, with their minimal parts and witnesses, in `code/data/p4_codex/`), with the SHA-256 values above. The computation also re-confirms $k = 7, 8$ (2 and 7 survivors, all UNSAT), independently of the hand proof. $\square$

5 Report for $9 \leq k \leq 16$

k	survivors	p4.codex (SAT)	p4.dfs (no SAT)	lists
9	6	all UNSAT	all UNSAT	identical
10	3	all UNSAT	all UNSAT	identical
11	137	all UNSAT	all UNSAT	identical
12	522	all UNSAT	all UNSAT	identical
13	4 395	all UNSAT	all UNSAT	identical
14	19 880	all UNSAT	all UNSAT	identical
15	155 890	all UNSAT	all UNSAT	identical
16	236 333	all UNSAT	partial: 17/24 blocks, all agree	identical

Decisions.

- *p4.codex*. Its decisions are complete for every $k \leq 16$, and every survivor is UNSAT. This includes 155 890 survivors at $k = 15$ and 236 333 at $k = 16$; there are no SAT and no undecided survivors.
- *p4.dfs*, $k = 15$. Its decisions are complete. The 155 890 survivors were decided by `sun.dfs.decide` (default mode) in 24 round-robin blocks, using the script `dfs.decide.chunk.py` of `p4.crosscheck`. The blocks were merged and checked by `merge.dfs.decisions.py`: every index was decided exactly once, and the survivor hash matches. All 155 890 are UNSAT. The run used 154 966 934 decision nodes and 4575.8 s of CPU summed over the blocks.
- *p4.dfs*, $k = 16$. This is only a **partial cross-check**: 17/24 blocks are finished, covering 167 404 survivors (state at compile time, 2026-09-26 15:52). All are UNSAT, in agreement with *p4.codex*. The run was then stopped; the remaining blocks are not claimed here.

The enumeration of $\mathcal{C}(k, k-1)$ by *p4.dfs* finished for both $k = 15$ and $k = 16$:

k	L_{k-1}	enum. nodes	gcd+NF	survivors	enum. time (s)	SHA-256 (prefix)
15	360360	13 266 631	2 992 302	155 890	420.3	8a4aca6d7ee2df8b
16	360360	26 169 762	5 314 669	236 333	776.8	2f8472ca4310cce9

Status for $13 \leq k \leq 16$ (report, not part of the theorem). The certified result of this cell is Theorem 1, for $k \leq 12$. The sizes $13 \leq k \leq 16$ are reported here and are *not* part of it. Under the stricter definition of Cell 5 (`p4.c5.tex`, §1: identical lists, every survivor decided with its minimal part, all files attached, a timed rerun under 10 minutes), the certified contiguous boundary is $K^* = 14$; the sizes $k = 15, 16$ are *decided, beyond the certified boundary*.

- For $k = 13, 14, 15$, both implementations enumerate identical survivor lists and both decide every survivor UNSAT; *p4.codex* also gives each survivor’s minimal part, in the files of `code/data/p4.codex/`.
- For $k = 16$ the survivor lists are identical and every survivor is UNSAT by *p4.codex*, with its minimal part. The independent *p4.dfs* decisions cover only 17/24 blocks (the run was stopped).

Comparison with O’Bryant (2007). K. O’Bryant, *On Z.-W. Sun’s disjoint congruence classes conjecture*, in *Combinatorial Number Theory*, de Gruyter 2007, pp. 403–411 (arXiv:math/0604347). Theorem 3 states: “The DCCC holds for $k \leq 20$. Moreover, a counterexample to the DCCC with minimal k does not have $k \in \{24, 30\}$.” For every size we completed, our result agrees with the first sentence. It was obtained by our own computation, not assumed. Two further points were checked:

- our values of L_{k-1} coincide with O’Bryant’s table of $L_k = \text{lcm}(1, \dots, k-1)$ for $k = 3, \dots, 13$;
- his example 20, 15, 12, 6, 6, 6, 6 for $k = 7$ passes his density test (Lemma 5). It appears among our two survivors for $k = 7$, and our decider finds it UNSAT.

The second sentence of his Theorem 3 is re-proved by hand in Cell 5 (`p4_c5.tex`).

A Survivors

Every survivor below was decided UNSAT. Survivors for $k \leq 6$: none.

$k = 7$: (6, 6, 6, 6, 6, 10, 15), (6, 6, 6, 6, 12, 15, 20).

$k = 8$: (6, 6, 6, 6, 6, 12, 21, 28), (6, 6, 6, 6, 6, 15, 21, 70), (6, 6, 6, 6, 6, 20, 28, 105), (6, 6, 6, 6, 10, 12, 28, 105), (6, 6, 6, 6, 12, 14, 20, 105), (6, 6, 6, 6, 12, 15, 21, 140), (6, 6, 6, 6, 12, 20, 28, 105).

$k = 9$: (6, 6, 6, 6, 6, 12, 40, 56, 105), (6, 6, 6, 6, 6, 14, 24, 40, 105), (6, 6, 6, 6, 6, 15, 21, 24, 280), (6, 6, 6, 6, 6, 20, 24, 56, 105), (6, 6, 6, 6, 6, 24, 28, 40, 105), (6, 6, 6, 6, 6, 24, 40, 56, 105).

$k = 10$: (6, 6, 6, 6, 6, 18, 24, 28, 40, 315), (6, 6, 6, 6, 6, 18, 24, 40, 56, 315), (6, 6, 6, 6, 6, 18, 24, 45, 63, 280).

$k = 11, 12, 13, 14$: 137, 522, 4395 and 19880 survivors, in `p4_dfs/results/k11_sun.canonical.json`, `k12_sun.canonical.json` and `k13_sun.canonical.json`, `k14_sun.canonical.json`. Each decision is in the corresponding `kXX_sun.json`. The full SHA-256 values are:

$k = 11$: `dee521791e4d6f13cbeaa1875c70c99e7278c8820143c0ab7fc0fe31003723fe`

$k = 12$: `d7ab51816f731ab2283f025ba0f31adff2efe79f0762d2ec03fc01ed2d30986d`

$k = 13$: `2dbfd65f26961615f44ddd6efced8fdf1d8489920b9de2bcfa111ab6b7411dd1`

$k = 14$: `f2ec65da7dedb65caec60c1acd7de28fc3bcb28a743a274dd240f32a038ed818`

$k = 15$: `8a4aca6d7ee2df8bbc4f278621f8216746990cf7c09164b452026bc0a2d87b40` (undecided)

$k = 16$: `2f8472ca4310cce90f832161c01520853a5c7b79516202f8d15ede27b81334fd` (undecided)